

DPDMS architecture

The Rushinga Provincial Disaster Monitoring and Management System: five hazard microservices (flood, drought, fire, zoonotic disease, mining accident), one gateway, one React frontend, an async alert pipeline, and a report service that reads live data without ever touching another service's database.

Spring Boot 3.3 / Java 17

Spring Cloud Gateway

Netflix Eureka

MySQL 8 - one schema/service

RabbitMQ

React 18 + Vite

JavaMail (SMTP)

WhatsApp Cloud API

01 Service topology - one entry point, one registry, isolated data

Every request, from every role, enters through the gateway. The gateway validates the JWT once and forwards the caller's identity as headers; each service still re-checks scoping itself. Eureka is how the gateway (and report-service) find a live instance by name - nobody hardcodes a host or port.

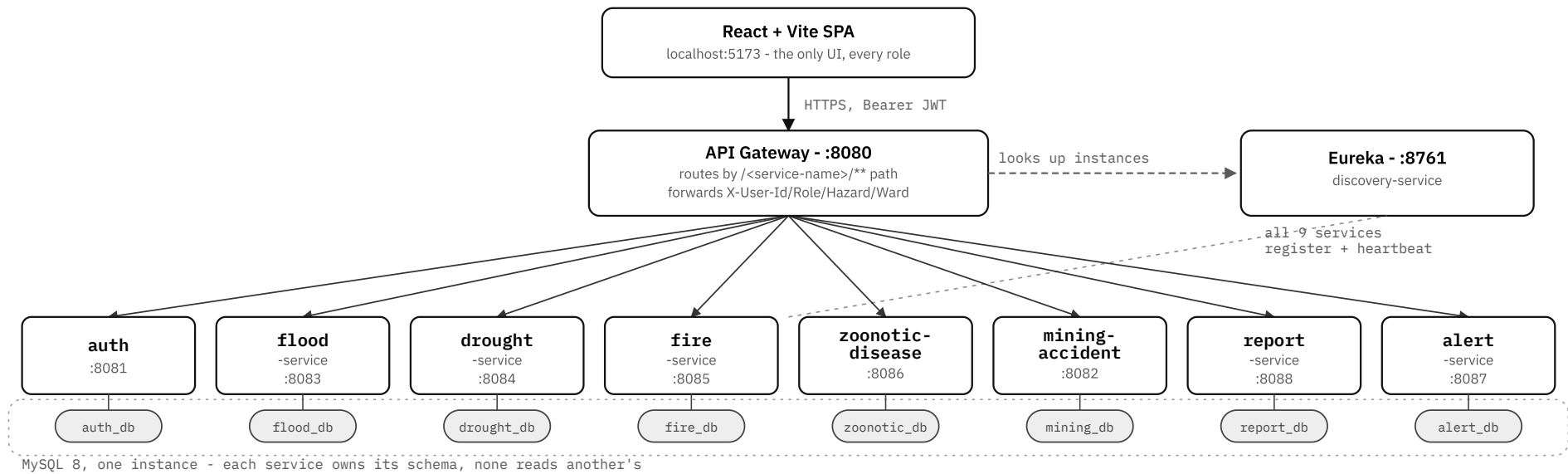


Figure 1 - `dashboard-service` is omitted here: since the React frontend took over its job, it registers with Eureka but has no routes or schema of its own any more.

02 Approval to alert - async, decoupled, never blocks

When a supervisor approves an incident, the hazard service commits the approval, then publishes an event and returns immediately - alert-service does the actual deciding and sending on its own time, on its own thread, from a separate consumer.



Figure 2 - if RabbitMQ or alert-service is down, incident capture and approval still succeed; the alert is simply delayed or logged as failed, per the brief's "degrade gracefully" requirement.

03 Reports read live data, never another service's database

report-service has no incident table of its own. For every report it calls each hazard's own GET /api/incidents directly (via Eureka, not through the gateway), forwarding the caller's identity headers unchanged - so a flood supervisor asking for a fire report is refused by fire-service itself, exactly as if they'd called it directly.

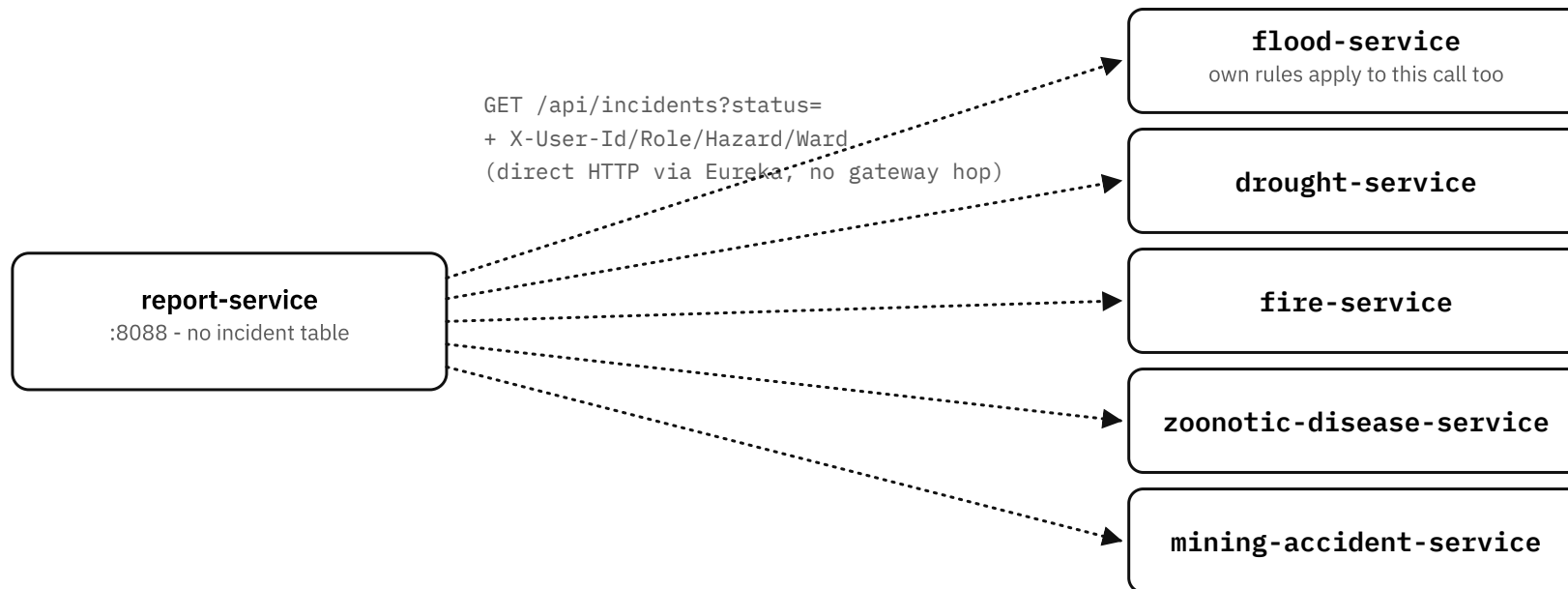


Figure 3 - the same call shape backs both the PDF/DOCX/XLSX/CSV writers and the React dashboard's live counts, trend chart and map: one code path, four report formats, and one page, all reading the same never-duplicated source of truth.

04 Who can do what

Enforced independently inside every hazard service via the shared HazardScopeGuard - never by the gateway or the frontend alone, per the brief's explicit requirement.

ROLE	SCOPE	CAN WRITE	SEES PENDING RECORDS
WARD_RECORDER	One (ward, hazard) pair	Create, edit own PENDING / CORRECTIONS_REQUESTED, delete own not-yet-approved	Only their own
PROVINCIAL_SUPERVISOR	One hazard, all wards	Approve / reject / request corrections, own hazard only	Their hazard's queue
NATIONAL_VIEWER	All 5 hazards	None - every write is 403	Never (APPROVED only)
PROVINCIAL_ADMIN	All 5 hazards	Read + choose report status/history	Yes, any hazard

solid, thin default request path - browser to gateway to service

dashed, thin Eureka service discovery / registration

solid, thick RabbitMQ async alert flow

dotted direct service-to-service call, bypasses the gateway (report-service)

gray tag a service's own MySQL schema